FI360 TYRE MODULE
DATABASE & DATA DICTIONARY

Fleet Intelligence 360 (FI360) — Version 1.0

Fleet Associates Africa Ltd (FAA)

10 August 2026

Document Control

Document ID	FI360-DDD-TYR-005
Version	1.0
Status	Baseline / Data Architecture
Parent	FI360 Tyre Module Software Requirements Specification v1.0
Core Foundation	FI360 Core Architecture & Common Services Specification v1.0
Standard	FI360 Module Development Standard v1.0
Module	FI360 Tyre Management
Owner	Fleet Associates Africa Ltd
Date	10 August 2026

1. Purpose

This document defines the physical data model and field-level data dictionary for the FI360 Tyre module. It translates the entities identified in the FI360 Tyre Module SRS (Section 28, Initial Data Model) into concrete, developer-ready table definitions, together with the identifier conventions, tenancy rules, audit columns, reference/lookup tables, relationships, indexing and constraint strategy required to implement them consistently with the FI360 Core Architecture and the FI360 Module Development Standard.

2. Scope

This document covers:

- The physical schema for all Tyre-module-owned tables (transactional and reference/lookup).
- Field-level data types, nullability, keys and descriptions for every table.
- The identifier, tenancy and audit-column conventions inherited from FI360 Core.
- Entity relationships, cardinality and referential-integrity rules.
- Indexing, constraint and data-integrity mechanisms that enforce the business rules in the SRS.
- Data retention and auditability treatment.

It does not define the REST/event API contracts (Document 06), UI/UX behaviour (Document 07), or the developer backlog and test plan (Document 08); those are addressed in the subsequent documents in this series.

3. Design Principles

- Module-owned data: the Tyre module owns and is authoritative for all tables in this document, per FI360 Core §17 (Data Architecture).
- No direct cross-module database coupling: other modules integrate through FI360 Core APIs/events, never by joining directly into Tyre tables.
- Every tenant-scoped table carries tenant_id and is filtered by it on every query; no request may cross tenant boundaries.
- Every physical tyre has exactly one FI360 Tyre ID for its entire life; the surrogate UUID primary key is internal, the FI360 Tyre ID is the external/business-facing identifier.
- History is never silently overwritten: transactional tables (fitment, inspection, movement, repair, retread, cost, disposition) are append-oriented; corrections use compensating rows, not in-place edits, once a row is closed/approved.
- PostgreSQL is the target relational database, per the SRS Recommended Technology Stack (Section 24); data access is via a typed ORM (Prisma or TypeORM) with version-controlled migrations.

4. Identifier & Naming Conventions

Table and column names use lower_snake_case. Every table has an internal UUID surrogate primary key (id) for joins and foreign keys, and — for entities that need one — an external, human-readable business identifier following the FI360 Core convention FI360-<MODULE>-<sequence>.

Identifier	Format	Example	Assigned by
FI360 Tyre ID	FI360-TYR-######	FI360-TYR-000321	Tyre module, on registration
FI360 Asset ID (referenced)	FI360-AST-######	FI360-AST-000123	Fleet Register (Core canonical asset reference)
FI360 Tenant ID (referenced)	FI360-TEN-######	FI360-TEN-000001	FI360 Core
FI360 Site ID (referenced)	FI360-SIT-######	FI360-SIT-000021	FI360 Core
FI360 User ID (referenced)	FI360-USR-######	FI360-USR-000045	FI360 Core

5. Standard Columns

Every table in this data dictionary includes the following standard columns unless explicitly noted otherwise. They are omitted from each table's own listing below and shown once here to avoid repetition.

Field	Data Type	Null	Key	Description
tenant_id	UUID	No	FK→core.tenant	Owning tenant. Present on every module-owned table; enforced at the query and row-level-security layer.
created_at	TIMESTAMPTZ	No		UTC timestamp the row was created.
created_by	UUID	No	FK→core.user	FI360 Core user (or service identity) that created the row.
updated_at	TIMESTAMPTZ	No		UTC timestamp of the last update.
updated_by	UUID	No	FK→core.user	FI360 Core user (or service identity) that last updated the row.

Field	Data Type	Null	Key	Description
row_version	INTEGER	No		Optimistic concurrency token; incremented on every update.
is_deleted	BOOLEAN	No		Soft-delete flag. Historical rows are never hard-deleted (Business Rule: history must not be silently overwritten).

6. Entity-Relationship Overview

TYRE MODULE ENTITY OVERVIEW

tyre_specification ← tyre → tyre_location (current state)

tyre → tyre_fitment (history) → core.asset (FI360-AST-######)

tyre → tyre_inspection → tyre_fitment (optional)

tyre → tyre_movement (history)

tyre → tyre_repair, tyre_retread, tyre_disposition

tyre_repair / tyre_retread / tyre_disposition → tyre_cost (consolidated ledger)

tyre → tyre_event_reference → FI360 Core Event Service

tyre is the hub entity. Specification data is normalised into tyre_specification so identical tyres share one catalogue row. Current state (location, active fitment) is held on tyre and its 1:1 state tables for fast lookup; full history is preserved in the append-only movement, fitment, inspection, repair, retread and cost tables. Disposition is a terminal 1:1 record. All financial events roll up into tyre_cost for lifecycle cost-per-kilometre reporting.

7. Reference Entities Owned by FI360 Core (Not Duplicated Here)

The following are referenced by foreign key from Tyre-module tables but are owned and maintained by FI360 Core or Fleet Register, per FI360 Core §17. The Tyre module stores only the reference (ID), never a duplicate copy of the underlying record.

Referenced Entity	Owner	Referenced From
core.tenant	FI360 Core	Every table (tenant_id)
core.site	FI360 Core	tyre, tyre_location, tyre_fitment, tyre_movement
core.user	FI360 Core	created_by/updated_by, inspector, actor, approver fields
core.asset	Fleet Register	tyre_fitment (asset_id)
core.supplier	FI360 Core	tyre, tyre_repair, tyre_retread
core.currency	FI360 Core	All cost/value fields

8. Tyre Module Data Dictionary

Each table below lists its module-specific fields. Standard tenancy and audit columns (Section 5) apply in addition unless a row explicitly overrides them (e.g. append-only history tables omit updated_at/updated_by/row_version, since rows are not mutated after creation).

8.1 tyre — Tyre Master Register

The tyre master register. One row represents one physical tyre for its entire life in the module. This is the module's primary source of truth referenced by FI360-TYR-###### and, where fitted, by the FI360 canonical Asset reference.

Field	Data Type	Null	Key	Description
id	UUID	No	PK	Internal surrogate primary key.
fi360_tyre_id	VARCHAR(20)	No	UNIQUE	Business identifier, format FI360-TYR-000001. Immutable once issued.
tenant_id	UUID	No	FK→core.tenant	Owning tenant. Enforced on every query per Core tenant-isolation rule.
site_id	UUID	Yes	FK→core.site	Current owning site/depot.
specification_id	UUID	No	FK→tyre_specification	Brand/pattern/size specification this tyre was manufactured to.
manufacturer_serial	VARCHAR(64)	Yes		Manufacturer-assigned serial number.
dot_production_code	VARCHAR(32)	Yes		DOT / production date code, captured where applicable.
barcode_qr	VARCHAR(64)	Yes	UNIQUE	Machine-readable FI360 identifier printed/affixed to the tyre.
tyre_type_id	UUID	No	FK→tyre_type	New / Retread / Reject classification at registration.
status_id	UUID	No	FK→tyre_status	Current lifecycle status (see Section 9, Enumerations).
current_location_id	UUID	Yes	FK→tyre_location	Pointer to the current location row (store/bay or fitted).
current_fitment_id	UUID	Yes	FK→tyre_fitment	Pointer to the active fitment row, where currently fitted.
retread_count	SMALLINT	No		Number of completed retread cycles. Monotonically non-decreasing (Business Rule).
supplier_id	UUID	Yes	FK→core.supplier	Originating supplier, if sourced externally.
purchase_date	DATE	Yes		Date of purchase/receipt into the register.
purchase_cost_amount	NUMERIC(14,2)	Yes		Original purchase cost.
currency_code	CHAR(3)	Yes	FK→core.currency	ISO 4217 currency of purchase_cost_amount.
created_at	TIMESTAMPTZ	No		UTC timestamp the row was created.
created_by	UUID	No	FK→core.user	FI360 Core user (or service identity) that created the row.
updated_at	TIMESTAMPTZ	No		UTC timestamp of the last update.

Field	Data Type	Null	Key	Description
updated_by	UUID	No	FK→core.user	FI360 Core user (or service identity) that last updated the row.
row_version	INTEGER	No		Optimistic concurrency token; incremented on every update.
is_deleted	BOOLEAN	No		Soft-delete flag. Historical rows are never hard-deleted (Business Rule: history must not be silently overwritten).

8.2 tyre_specification — Tyre Specification Catalogue

A shared, reusable specification (brand, pattern, size, construction) that many physical tyre rows reference. Reduces duplication and standardises reporting across identical tyre types.

Field	Data Type	Null	Key	Description
id	UUID	No	PK	Internal surrogate primary key.
tenant_id	UUID	No	FK→core.tenant	Owning tenant (specification catalogues are tenant-scoped, not shared across tenants).
brand	VARCHAR(64)	No		Manufacturer brand, e.g. Bridgestone.
pattern_model	VARCHAR(64)	No		Manufacturer pattern/model, e.g. R150.
size	VARCHAR(32)	No		Tyre size designation, e.g. 315/80R22.5.
construction_type	VARCHAR(16)	No		Radial / Bias.
load_index	VARCHAR(8)	Yes		Manufacturer load index rating.
speed_rating	VARCHAR(4)	Yes		Manufacturer speed rating.
tread_pattern	VARCHAR(64)	Yes		Named tread pattern where distinct from pattern_model.
tread_depth_new_mm	NUMERIC(4,1)	Yes		Original tread depth when new, in millimetres.
recommended_position_type_id	UUID	Yes	FK→position_type	Recommended fitment position: Steer / Drive / Trailer / Other.
is_active	BOOLEAN	No		Whether this specification may still be used for new registrations.
created_at	TIMESTAMPTZ	No		UTC timestamp the row was created.
created_by	UUID	No	FK→core.user	FI360 Core user (or service identity) that created the row.
updated_at	TIMESTAMPTZ	No		UTC timestamp of the last update.
updated_by	UUID	No	FK→core.user	FI360 Core user (or service identity) that last updated the row.
row_version	INTEGER	No		Optimistic concurrency token; incremented on every update.

Field	Data Type	Null	Key	Description
is_deleted	BOOLEAN	No		Soft-delete flag. Historical rows are never hard-deleted (Business Rule: history must not be silently overwritten).

8.3 tyre_location — Current Location State

Current, single-row-per-tyre location state (store, bay/bin, or fitted). Historical movement between locations is recorded separately in tyre_movement so this table always reflects only the present position.

Field	Data Type	Null	Key	Description
id	UUID	No	PK	Internal surrogate primary key.
tenant_id	UUID	No	FK→core.tenant	Owning tenant.
tyre_id	UUID	No	FK→tyre, UNIQUE	The tyre this location state describes (one active row per tyre).
location_type	VARCHAR(16)	No		STORE / FITTED / IN_TRANSIT / EXTERNAL (e.g. at a repairer).
site_id	UUID	Yes	FK→core.site	Site/depot currently holding the tyre.
store_code	VARCHAR(32)	Yes		Store or warehouse code within the site.
bay_bin	VARCHAR(32)	Yes		Bay/bin/shelf reference within the store.
status	VARCHAR(16)	No		IN_STORE / ISSUED / FITTED / QUARANTINE / IN_TRANSIT.
effective_from	TIMESTAMPTZ	No		When this location state took effect.
created_at	TIMESTAMPTZ	No		UTC timestamp the row was created.
created_by	UUID	No	FK→core.user	FI360 Core user (or service identity) that created the row.
updated_at	TIMESTAMPTZ	No		UTC timestamp of the last update.
updated_by	UUID	No	FK→core.user	FI360 Core user (or service identity) that last updated the row.
row_version	INTEGER	No		Optimistic concurrency token; incremented on every update.
is_deleted	BOOLEAN	No		Soft-delete flag. Historical rows are never hard-deleted (Business Rule: history must not be silently overwritten).

8.4 tyre_fitment — Fitment History

One row per fit/remove cycle of a tyre on a vehicle/asset position. Enforces the rule that a tyre cannot be actively fitted to two positions, and a position cannot hold two active tyres, at the same time.

Field	Data Type	Null	Key	Description
id	UUID	No	PK	Internal surrogate primary key.
tenant_id	UUID	No	FK→core.tenant	Owning tenant.

Field	Data Type	Null	Key	Description
tyre_id	UUID	No	FK→tyre	Tyre being fitted.
asset_id	VARCHAR(20)	No	FK→core.asset (FI360-AST-######)	Canonical FI360 Asset reference (vehicle/trailer/equipment), owned by Fleet Register.
position_code	VARCHAR(16)	No		Axle/position code, e.g. AXLE1-LEFT-OUTER.
position_type_id	UUID	No	FK→position_type	Steer / Drive / Trailer / Other.
fitted_date	TIMESTAMPTZ	No		Date/time the tyre was fitted.
fitted_odometer	INTEGER	Yes		Vehicle odometer reading (km) at fitment.
fitted_hours	NUMERIC(10,1)	Yes		Equipment hour-meter reading at fitment, where applicable.
fitted_by_user_id	UUID	No	FK→core.user	User who performed the fitment.
removed_date	TIMESTAMPTZ	Yes		Date/time the tyre was removed. NULL while active.
removed_odometer	INTEGER	Yes		Odometer reading at removal.
removed_by_user_id	UUID	Yes	FK→core.user	User who performed the removal.
removal_reason_id	UUID	Yes	FK→movement_reason	Reason for removal (rotation, damage, wear, disposal, etc.).
is_active	BOOLEAN	No		TRUE while fitted; set FALSE on removal. Unique partial index enforces one active row per tyre and per (asset_id, position_code).
created_at	TIMESTAMPTZ	No		UTC timestamp the row was created.
created_by	UUID	No	FK→core.user	FI360 Core user (or service identity) that created the row.
updated_at	TIMESTAMPTZ	No		UTC timestamp of the last update.
updated_by	UUID	No	FK→core.user	FI360 Core user (or service identity) that last updated the row.
row_version	INTEGER	No		Optimistic concurrency token; incremented on every update.
is_deleted	BOOLEAN	No		Soft-delete flag. Historical rows are never hard-deleted (Business Rule: history must not be silently overwritten).

8.5 tyre_inspection — Inspection & Condition History

One row per inspection event, whether performed in-store or while fitted. Supports tread-depth trending, pressure monitoring and condition/damage capture.

Field	Data Type	Null	Key	Description
id	UUID	No	PK	Internal surrogate primary key.
tenant_id	UUID	No	FK→core.tenant	Owning tenant.

Field	Data Type	Null	Key	Description
tyre_id	UUID	No	FK→tyre	Tyre inspected.
fitment_id	UUID	Yes	FK→tyre_fitment	Related active fitment, where inspected in-service.
inspection_date	TIMESTAMPTZ	No		Date/time of inspection.
inspector_user_id	UUID	No	FK→core.user	User who performed the inspection.
tread_depth_inner_mm	NUMERIC(4,1)	Yes		Inner tread depth reading, millimetres.
tread_depth_center_mm	NUMERIC(4,1)	Yes		Centre tread depth reading, millimetres.
tread_depth_outer_mm	NUMERIC(4,1)	Yes		Outer tread depth reading, millimetres.
pressure_kpa	NUMERIC(6,1)	Yes		Measured tyre pressure, kilopascals.
condition_code_id	UUID	No	FK→condition_code	Overall condition classification (Good/Fair/Poor/Damaged/etc.).
damage_found	BOOLEAN	No		Whether damage was identified during inspection.
findings_notes	TEXT	Yes		Free-text findings.
recommended_action	VARCHAR(32)	Yes		None / Monitor / Repair / Retread / Remove / Scrap.
created_at	TIMESTAMPTZ	No		UTC timestamp the row was created.
created_by	UUID	No	FK→core.user	FI360 Core user (or service identity) that created the row.
updated_at	TIMESTAMPTZ	No		UTC timestamp of the last update.
updated_by	UUID	No	FK→core.user	FI360 Core user (or service identity) that last updated the row.
row_version	INTEGER	No		Optimistic concurrency token; incremented on every update.
is_deleted	BOOLEAN	No		Soft-delete flag. Historical rows are never hard-deleted (Business Rule: history must not be silently overwritten).

8.6 tyre_movement — Movement History

Immutable append-only log of every location change for a tyre. tyre_location always reflects the most recent row of this log for that tyre.

Field	Data Type	Null	Key	Description
id	UUID	No	PK	Internal surrogate primary key.
tenant_id	UUID	No	FK→core.tenant	Owning tenant.
tyre_id	UUID	No	FK→tyre	Tyre that moved.
movement_date	TIMESTAMPTZ	No		Date/time of the movement.
from_location_type	VARCHAR(16)	Yes		Location type moved from.
from_site_id	UUID	Yes	FK→core.site	Site moved from.

Field	Data Type	Null	Key	Description
from_bay_bin	VARCHAR(32)	Yes		Bay/bin moved from.
to_location_type	VARCHAR(16)	No		Location type moved to.
to_site_id	UUID	Yes	FK→core.site	Site moved to.
to_bay_bin	VARCHAR(32)	Yes		Bay/bin moved to.
reason_id	UUID	No	FK→movement_reason	Reason for the movement.
actor_user_id	UUID	No	FK→core.user	User (or service identity) that performed/triggered the movement.
reference_document	VARCHAR(64)	Yes		Related document reference, e.g. goods-receipt or transfer note number.
created_at	TIMESTAMPTZ	No		UTC timestamp the row was created.
created_by	UUID	No	FK→core.user	FI360 Core user (or service identity) that created the row.
is_deleted	BOOLEAN	No		Soft-delete flag. Historical rows are never hard-deleted (Business Rule: history must not be silently overwritten).

8.7 tyre_repair — Repair History

One row per repair event performed on a tyre (excluding full retreading, which is tracked in tyre_retread).

Field	Data Type	Null	Key	Description
id	UUID	No	PK	Internal surrogate primary key.
tenant_id	UUID	No	FK→core.tenant	Owning tenant.
tyre_id	UUID	No	FK→tyre	Tyre repaired.
repair_date	DATE	No		Date of repair.
repair_type_id	UUID	No	FK→repair_type	Type of repair performed, e.g. puncture plug, section repair.
supplier_id	UUID	Yes	FK→core.supplier	Repairer/supplier performing the work.
cost_amount	NUMERIC(14,2)	Yes		Cost of the repair.
currency_code	CHAR(3)	Yes	FK→core.currency	ISO 4217 currency of cost_amount.
result_code	VARCHAR(16)	No		SUCCESSFUL / FAILED / NOT_REPAIRABLE.
warranty_flag	BOOLEAN	No		Whether the repair was covered under warranty.
notes	TEXT	Yes		Free-text notes.
created_at	TIMESTAMPTZ	No		UTC timestamp the row was created.
created_by	UUID	No	FK→core.user	FI360 Core user (or service identity) that created the row.
updated_at	TIMESTAMPTZ	No		UTC timestamp of the last update.
updated_by	UUID	No	FK→core.user	FI360 Core user (or service identity) that last updated the row.

Field	Data Type	Null	Key	Description
row_version	INTEGER	No		Optimistic concurrency token; incremented on every update.
is_deleted	BOOLEAN	No		Soft-delete flag. Historical rows are never hard-deleted (Business Rule: history must not be silently overwritten).

8.8 tyre_retread — Retread Cycle History

One row per retread cycle sent to and received from a retreading supplier. tyre.retread_count reflects the count of successfully completed rows here and cannot decrease through ordinary transactions.

Field	Data Type	Null	Key	Description
id	UUID	No	PK	Internal surrogate primary key.
tenant_id	UUID	No	FK→core.tenant	Owning tenant.
tyre_id	UUID	No	FK→tyre	Tyre retreaded.
retread_cycle_number	SMALLINT	No		Sequential cycle number for this tyre (1, 2, 3…).
supplier_id	UUID	No	FK→core.supplier	Retreading supplier.
sent_date	DATE	No		Date sent for retreading.
received_date	DATE	Yes		Date received back. NULL while at supplier.
cost_amount	NUMERIC(14,2)	Yes		Cost of the retread cycle.
currency_code	CHAR(3)	Yes	FK→core.currency	ISO 4217 currency of cost_amount.
result_code	VARCHAR(16)	Yes		SUCCESSFUL / REJECTED / SCRAPPED_AT_SUPPLIER.
new_tread_pattern	VARCHAR(64)	Yes		Tread pattern applied, if changed from original.
new_tread_depth_mm	NUMERIC(4,1)	Yes		Tread depth of the retreaded casing on return.
created_at	TIMESTAMPTZ	No		UTC timestamp the row was created.
created_by	UUID	No	FK→core.user	FI360 Core user (or service identity) that created the row.
updated_at	TIMESTAMPTZ	No		UTC timestamp of the last update.
updated_by	UUID	No	FK→core.user	FI360 Core user (or service identity) that last updated the row.
row_version	INTEGER	No		Optimistic concurrency token; incremented on every update.
is_deleted	BOOLEAN	No		Soft-delete flag. Historical rows are never hard-deleted (Business Rule: history must not be silently overwritten).

8.9 tyre_cost — Consolidated Cost Ledger

Consolidated, queryable cost ledger for cost-per-kilometre and lifecycle-cost reporting. Populated from purchase, repair, retread and disposition events; not itself the origin of a cost, so it references the originating transaction.

Field	Data Type	Null	Key	Description
id	UUID	No	PK	Internal surrogate primary key.
tenant_id	UUID	No	FK→core.tenant	Owning tenant.
tyre_id	UUID	No	FK→tyre	Tyre this cost applies to.
cost_type_id	UUID	No	FK→cost_type	PURCHASE / REPAIR / RETREAD / FITMENT_LABOUR / DISPOSAL / OTHER.
amount	NUMERIC(14,2)	No		Cost amount.
currency_code	CHAR(3)	No	FK→core.currency	ISO 4217 currency of amount.
cost_date	DATE	No		Date the cost was incurred.
reference_table	VARCHAR(32)	Yes		Originating table, e.g. tyre_repair, tyre_retread, tyre_disposition.
reference_id	UUID	Yes		Originating row's primary key.
notes	TEXT	Yes		Free-text notes.
created_at	TIMESTAMPTZ	No		UTC timestamp the row was created.
created_by	UUID	No	FK→core.user	FI360 Core user (or service identity) that created the row.
is_deleted	BOOLEAN	No		Soft-delete flag. Historical rows are never hard-deleted (Business Rule: history must not be silently overwritten).

8.10 tyre_disposition — Final Disposition

Final-state record for a tyre leaving the fleet (scrap, sale, trade-in, casing return). A tyre with a disposition row cannot return to service without an approved exceptional workflow.

Field	Data Type	Null	Key	Description
id	UUID	No	PK	Internal surrogate primary key.
tenant_id	UUID	No	FK→core.tenant	Owning tenant.
tyre_id	UUID	No	FK→tyre, UNIQUE	Tyre disposed of (one disposition per tyre under ordinary process).
disposition_date	DATE	No		Date of disposition.
reason_id	UUID	No	FK→disposition_reason	Reason for disposition, e.g. worn out, irreparable damage, end of retread life.
disposal_value	NUMERIC(14,2)	Yes		Sale/scrap/trade-in value received, if any.
currency_code	CHAR(3)	Yes	FK→core.currency	ISO 4217 currency of disposal_value.
destination	VARCHAR(64)	Yes		Buyer/scrap-handler/destination description.
approved_by_user_id	UUID	No	FK→core.user	User who approved the disposition.
notes	TEXT	Yes		Free-text notes.

Field	Data Type	Null	Key	Description
created_at	TIMESTAMPTZ	No		UTC timestamp the row was created.
created_by	UUID	No	FK→core.user	FI360 Core user (or service identity) that created the row.
updated_at	TIMESTAMPTZ	No		UTC timestamp of the last update.
updated_by	UUID	No	FK→core.user	FI360 Core user (or service identity) that last updated the row.
row_version	INTEGER	No		Optimistic concurrency token; incremented on every update.
is_deleted	BOOLEAN	No		Soft-delete flag. Historical rows are never hard-deleted (Business Rule: history must not be silently overwritten).

8.11 tyre_event_reference — Outbound Event Log

Record of domain events published by the Tyre module to the FI360 Core Event Service for consumption by other modules (Workshop, Fleet, Analytics). Supports replay, correlation and audit of module-to-module communication.

Field	Data Type	Null	Key	Description
id	UUID	No	PK	Internal surrogate primary key.
tenant_id	UUID	No	FK→core.tenant	Owning tenant.
tyre_id	UUID	Yes	FK→tyre	Related tyre, where the event concerns a specific tyre.
event_id	UUID	No	UNIQUE	Unique event identifier as published to the Core Event Service.
event_type	VARCHAR(64)	No		e.g. TYRE_FITTED, TYRE_REMOVED, TYRE_SCRAPPED, TYRE_INSPECTION_RECORDED.
correlation_id	UUID	No		Correlation ID linking related events/transactions across modules.
payload_ref	VARCHAR(128)	Yes		Pointer to the full event payload (event store or object reference).
published_at	TIMESTAMPTZ	Yes		Timestamp the event was successfully published. NULL while pending.
status	VARCHAR(16)	No		PENDING / PUBLISHED / FAILED.
created_at	TIMESTAMPTZ	No		UTC timestamp the row was created.
created_by	UUID	No	FK→core.user	FI360 Core user (or service identity) that created the row.
is_deleted	BOOLEAN	No		Soft-delete flag. Historical rows are never hard-deleted (Business Rule: history must not be silently overwritten).

8.12 Reference / Lookup Tables

Controlled code tables used across the module. Each follows the common lookup shape (id, tenant_id [nullable for global defaults], code, label, sort_order/severity, plus standard audit columns) unless noted. Per FI360 Core §8, historical transactions retain the meaning of the code at the time they were recorded even if a code's label is later changed.

Table	Purpose / Typical Values	Additional Columns
tyre_status	Registered, In Store, Issued, Fitted, Under Repair, At Retreader, Quarantined, Scrapped, Disposed.	id, tenant_id (nullable — global defaults with tenant override), code, label, is_terminal, sort_order
tyre_type	New, Retread, Reject.	id, tenant_id, code, label, sort_order
position_type	Steer, Drive, Trailer, Other.	id, tenant_id, code, label, sort_order
movement_reason	Rotation, Damage, Wear, Repair Dispatch, Retread Dispatch, Return to Store, Disposal, Other.	id, tenant_id, code, label, sort_order
repair_type	Puncture Plug, Section Repair, Valve Replacement, Other.	id, tenant_id, code, label, sort_order
cost_type	Purchase, Repair, Retread, Fitment Labour, Disposal, Other.	id, tenant_id, code, label, sort_order
disposition_reason	Worn Out, Irreparable Damage, Retread Life Exhausted, Accident Loss, Theft, Other.	id, tenant_id, code, label, sort_order
condition_code	Good, Fair, Poor, Damaged, Non-Repairable.	id, tenant_id, code, label, severity_rank

9. Enumerations and Status Lifecycle

tyre.status_id drives the tyre lifecycle. The standard progression, per SRS Section 14 (Tyre Lifecycle) and Section 29 (Business Rules), is:

TYRE LIFECYCLE

Registered → In Store → Issued → Fitted ⇄Removed → (Inspection / Repair / Retread) → back to In Store

→ Scrapped / Disposed (terminal — requires approved exceptional workflow to reverse)

- Only a tyre in an eligible status (e.g. In Store) may transition to Fitted.
- A tyre cannot hold two simultaneous active tyre_fitment rows (is_active = TRUE); enforced by a unique partial index on (tyre_id) WHERE is_active.
- A vehicle position cannot hold two simultaneous active fitments unless explicitly configured; enforced by a unique partial index on (asset_id, position_code) WHERE is_active.
- Removal sets the corresponding tyre_fitment.is_active = FALSE and stamps removed_date/removed_odometer/removed_by_user_id; it never deletes the row.
- Scrapped/Disposed is terminal: tyre_disposition existing for a tyre_id blocks any new tyre_fitment insert unless an approved exception workflow explicitly reactivates the tyre (application-layer control, logged to tyre_event_reference).

10. Relationships and Cardinality

Relationship	Cardinality	Notes
tyre_specification → tyre	1 : many	One specification is shared by many physical tyres.
tyre → tyre_location	1 : 1	Current-state table; upserted on every movement.
tyre → tyre_fitment	1 : many	Full fit/remove history; at most one active row at a time.
core.asset → tyre_fitment	1 : many	A vehicle position accumulates many fitment rows over time.
tyre → tyre_inspection	1 : many	Unbounded inspection history.
tyre_fitment → tyre_inspection	1 : many (optional)	Inspections may or may not relate to an active fitment.
tyre → tyre_movement	1 : many	Append-only movement log.
tyre → tyre_repair	1 : many	
tyre → tyre_retread	1 : many	Cycle-numbered; retread_cycle_number strictly increasing per tyre.
tyre → tyre_cost	1 : many	Populated from repair, retread, disposition and purchase events.
tyre → tyre_disposition	1 : 1 (0 or 1)	Terminal record; at most one under ordinary process.
tyre → tyre_event_reference	1 : many	One row per outbound domain event concerning the tyre.

11. Indexing Strategy

- Every table: composite index on (tenant_id, id) supporting row-level tenant isolation.
- tyre: unique index on fi360_tyre_id; unique index on barcode_qr (where not null); index on (tenant_id, status_id) for operational status queries.
- tyre_fitment: unique partial index on (tyre_id) WHERE is_active = TRUE; unique partial index on (asset_id, position_code) WHERE is_active = TRUE; index on (asset_id, fitted_date) for vehicle history queries.
- tyre_inspection, tyre_movement: index on (tyre_id, inspection_date/movement_date DESC) for trend and history queries.
- tyre_cost: index on (tyre_id, cost_date) for lifecycle cost-per-kilometre calculations; index on cost_type_id for cost-category reporting.
- tyre_retread: unique index on (tyre_id, retread_cycle_number).

12. Constraints and Data Integrity Rules

SRS Business Rule	Database Enforcement
Only an eligible tyre can be fitted.	Application-layer check against tyre.status_id at the API/service layer prior to INSERT into tyre_fitment; not enforceable by a simple CHECK constraint alone.

SRS Business Rule	Database Enforcement
A tyre cannot be actively fitted to two positions at once.	Unique partial index on tyre_fitment(tyre_id) WHERE is_active = TRUE.
A position cannot have two active tyres unless configured.	Unique partial index on tyre_fitment(asset_id, position_code) WHERE is_active = TRUE, with a configuration flag to relax where explicitly required.
Removal closes the active fitment relationship.	Service-layer transaction: UPDATE tyre_fitment SET is_active = FALSE, removed_* ... in the same transaction as any new fitment; never a DELETE.
Historical transactions must not be silently overwritten.	History tables (fitment, inspection, movement, repair, retread, cost) are insert-only from the application; UPDATE is restricted to closing-fields only (e.g. removed_date) via a defined set of service methods, not open field edits.
Closed/approved transactions require controlled correction.	Corrections are new compensating rows referencing the original via reference_id/notes, not edits to the closed row.
Retread count cannot decrease through ordinary transactions.	Trigger or application check on tyre.retread_count preventing UPDATE where new value < current value, except via the approved exception workflow.
Scrapped/retired tyres cannot return to service without an approved exceptional workflow.	Service-layer guard: INSERT into tyre_fitment is blocked while a tyre_disposition row exists for the tyre_id, unless an authorised reactivation event is logged to tyre_event_reference.
Tenant ownership must be preserved.	NOT NULL tenant_id with foreign key to core.tenant on every table; row-level security policy filtering by session tenant context.
Material lifecycle transitions must be auditable.	Every status/location/fitment change writes to tyre_movement and/or tyre_event_reference in the same transaction as the state change.

13. Auditability and History

- Every table carries created_at/created_by and, where mutable, updated_at/updated_by and row_version for optimistic concurrency.
- Append-only history tables (tyre_movement, tyre_cost, tyre_event_reference) omit updated_at/updated_by/row_version by design — a row, once written, is not modified.
- Soft delete only: is_deleted marks a row as logically removed; hard DELETE is not permitted on Tyre-module tables in production.
- Material lifecycle transitions (status change, fitment, removal, disposition) additionally publish a corresponding row to tyre_event_reference for cross-module traceability and Core audit-platform consumption, per FI360 Core §14 (Audit and Observability).

14. Data Retention

Tyre lifecycle and cost history are retained for the life of the tenant relationship as an operational and audit requirement; no automatic purge of tyre history is performed. Soft-deleted reference/lookup rows are retained to preserve the historical meaning of codes used on closed transactions, consistent with FI360 Core §8 (Common Reference Data). Backup, restore and RPO/RTO targets follow FI360 Core §20 (Backup, Recovery and Availability).

15. Acceptance Criteria

Area	Acceptance Condition
Identity	Every tyre resolves to exactly one fi360_tyre_id and one internal id for its full lifecycle.
Tenancy	No query can return rows for a tenant_id other than the authenticated session's tenant.
Fitment integrity	The database rejects (via unique partial index) any attempt to create a second simultaneous active fitment for the same tyre or the same asset/position.
History integrity	No history-table row can be hard-deleted or have its historical fields altered outside the defined correction pattern.
Cost traceability	Every row in tyre_cost resolves via reference_table/reference_id to a specific originating transaction.
Event traceability	Every material lifecycle transition produces a corresponding tyre_event_reference row with a valid correlation_id.

16. Next Document

DOCUMENT 06 — FI360 TYRE API & INTEGRATION CONTRACT will define the REST/OpenAPI contract and event schemas that expose the entities in this data dictionary to the FI360 Core API Gateway and Event Service, enabling controlled integration with Workshop, Fleet, Parts, Telematics and Analytics.

FI360 Tyre principle: every tyre has one identity, one traceable lifecycle and one auditable history.